Real-Time American Sign Language Detection Using MediaPipe Hand Localization and ResNet-50 Classification

Udit Asthana
Manipal Institute of Technology
Reg. No: 240905310
Roll No: 31

Abhishek L Prabhu
Manipal Institute of Technology
Reg. No: 240905676
Roll No: 71

Ashwath Patil
Manipal Institute of Technology
Reg. No: 240905460
Roll No: 46

Kishan Kanvathirtha
Manipal Institute of Technology
Reg. No: 240905678
Roll No: 72

Arnav Dugad
Manipal Institute of Technology
Reg. No: 230905227
Roll No: 73

Abstract—American Sign Language (ASL) is the primary communication mode for the hearing-impaired community. Real-time automated recognition of ASL gestures remains an open engineering problem. This paper describes a two-stage pipeline: hand region localization via MediaPipe, followed by letter classification with a fine-tuned ResNet-50 backbone and a custom head. Training uses the publicly available ASL Alphabet dataset from Kaggle, evaluated over 26 letter classes. A cosine annealing scheduler with linear warm-up, gradient clipping, and early stopping are applied during training. Test accuracy: 99.80%. The system runs in real-time on a live webcam feed. Hand skeleton overlays and predicted letter labels are rendered per frame. A FastAPI inference server and browser frontend enable interactive deployment. Landmark-based detection combined with CNN classification is viable for low-latency sign language recognition.

Index Terms—sign language recognition, ASL, MediaPipe, ResNet-50, hand detection, real-time inference, deep learning, convolutional neural networks

I. Introduction

Sign language is the primary communication mode for over 70 million deaf and hard-of-hearing people worldwide [1]. ASL specifically — used mostly in North America — uses hand gestures and finger-spelling to express words and concepts. Despite how widely it's used, there are almost no real-time automated translation tools available. That gap makes everyday communication with the hearing majority significantly harder than it needs to be.

CNNs and real-time object detection frameworks have opened up new options for gesture recognition. Systems that can classify a gesture from a live video frame in milliseconds are a meaningful step toward accessible communication technology. Our approach uses two stages: first, a MediaPipe hand tracking module localizes the hand in each camera frame, producing a tight bounding box crop (not visible to the user) and a skeleton overlay (visible). Second, a fine-tuned ResNet-50 [3] classifies the cropped image into one of 26 ASL alphabet letters. The result is overlaid on the live stream in real time.

Our contributions:

- A modular training pipeline with configurable optimizer, scheduler, and early stopping, implemented in PyTorch.
- MediaPipe-based hand detection integrated with a ResNet-50 classification head for end-to-end ASL letter recognition.
- A real-time inference module with webcam integration and visual feedback.
- A FastAPI web interface for browser-based interaction with the live pipeline.

II. Related Work

Early sign language recognition systems depended on sensor-based hardware: data gloves, depth cameras, 3D pose capture [4]. These methods worked. They also imposed hardware constraints that blocked practical deployment.

Deep learning shifted the dominant approach to image-based methods. Convolutional networks trained on static hand images showed strong performance on isolated gesture classification [5]. Transfer learning from ImageNet-pretrained backbones — VGG-16, ResNet, MobileNet — extended this to smaller, domain-specific datasets with improved accuracy.

Real-time landmark-based hand tracking has been applied to gesture spotting. End-to-end two-stage systems covering the full ASL alphabet in a live-camera setting are less common. This work targets that gap.

III. Dataset

The ASL Alphabet dataset [6] is sourced from Kaggle. It contains 87,000 labeled images across 29 classes: 26

letters (A–Z), plus SPACE, DELETE, and NOTHING. Image dimensions: 200×200 pixels, RGB.

Images are resized to 224×224 to match the ResNet-50 input specification. Normalization uses standard ImageNet mean and standard deviation. The dataset is partitioned randomly: 70% training, 20% validation, 10% test. Random splitting preserves class balance across splits.

A _TransformSubset wrapper applies separate transforms to each split. Training images receive random horizontal flipping and color jitter. Validation and test images receive only deterministic resize and normalization. Batch size is 128. Shuffling is enabled for training and disabled for evaluation.

Approximate split sizes: 60,900 training samples, 17,400 validation samples, 8,700 test samples.

IV. Methodology

A. System Overview

Inference proceeds in two sequential stages.

1) Hand Detection (MediaPipe): Each video frame is passed to MediaPipe. Bounding boxes are returned around detected hand regions.
2) Letter Classification (ResNet-50): The hand crop is extracted, resized, and forwarded to the classifier. A probability distribution over 26 ASL letter classes is returned.

The predicted letter and bounding box are overlaid on the original frame.

B. Hand Detection Module

The HandDetector class wraps the MediaPipe HandLandmarker Tasks API. The operating mode is VIDEO. This mode enables frame-to-frame tracking through MediaPipe's internal Kalman filter.

The Kalman filter requires strictly monotonically increasing real wall-clock millisecond timestamps. Passing bare integer counters (1, 2, 3, …) is a common implementation error. It mis-models velocity. Landmark jitter increases. Timestamps here are derived from time.monotonic_ns().

Detection thresholds are raised above MediaPipe defaults. Minimum hand detection and presence confidence: 0.65 (default: 0.50). Tracking confidence: 0.55. The defaults are too permissive for static-sign ASL. Partially-occluded hands during letter transitions produce false detections under the defaults.

Per-frame output is a list of up to two hand results. For each hand, the raw landmark bounding box is computed from the min/max of all 21 normalized landmark coordinates. The box is expanded by 35% per dimension (expansion=0.35). This provides context around the hand edges. The expanded box is smoothed across frames via exponential smoothing: $0.3 \cdot \text{prev} + 0.7 \cdot \text{new}$.

Motion magnitude is computed on the raw (pre-smoothing) center against the previous smoothed center. This ordering is deliberate. Computing motion after smoothing causes fast-moving hands to appear stable — the smoothed center barely shifts, defeating the stability gate. Motion is measured as Euclidean (L2) distance. Manhattan (L1) distance under-reports diagonal motion by approximately 30%.

Per-hand tracking state is keyed by hand index. A single shared variable fails silently when two hands appear simultaneously in frame. When a hand disappears, its tracking state is evicted. The next detection initializes from scratch. Zero-area ROI crops after boundary clamping are discarded; they are not passed to the classifier.

When multiple hands are detected at inference time, the detection with the highest handedness classifier score is selected:

$$\text{det}^* = \arg\max_{d \in \mathcal{D}} d.\text{score} \tag{1}$$

The HandDetection dataclass carries per hand: smoothed pixel bbox (x_1, y_1, x_2, y_2), handedness label and score, 21 raw normalized landmarks, BGR ROI crop, boolean stability flag, raw motion magnitude in pixels.

C. Classification Model (SLD)

The Sign Language Detector (SLD) uses a ResNet-50 backbone. All backbone parameters are frozen. A trainable classification head is appended.

The backbone's final average pooling layer outputs a 2048-dimensional feature vector. The original fully-connected layer is discarded entirely. The replacement head is:

$$\hat{y} = \text{head}(\text{backbone}(x)) \tag{2}$$

$$\text{head}(z) = W_2 \cdot \text{GELU}(\text{Dropout}(\text{LayerNorm}(z) \cdot W_1)) \tag{3}$$

$W_1 \in \mathbb{R}^{2048 \times 512}$, $W_2 \in \mathbb{R}^{512 \times 29}$. Output covers 29 classes: 26 letters plus del, nothing, space.

LayerNorm is used at the backbone-to-head boundary. BatchNorm is not. BatchNorm is sensitive to batch size. Its behavior differs between train and eval modes. With a frozen backbone and a trainable head only, this instability is unacceptable. LayerNorm normalizes per-sample. It is invariant to batch size.

GELU is used in place of ReLU. GELU is smooth. Its gradient is non-zero at negative values. Dropout at $p = 0.3$ regularizes the intermediate 512-dimensional representation.

At inference, the checkpoint is loaded via torch.load with weights_only=False. This is explicit, not incidental. The checkpoint stores optimizer and scheduler state alongside model weights. It cannot be loaded in weights-only mode. The flag also suppresses the PyTorch 2.x deprecation warning and documents intent.

D. Training Configuration

Training configuration is encapsulated in a dataclass (TRAINING_CONFIG). All hyperparameters are stored there. Runtime overrides are accepted via command-line arguments. Key settings are listed in Table I.

The dataset is partitioned via random_split: 70% training, 20% validation, 10% test. A _TransformSubset wrapper is applied to each split. This allows training and evaluation transforms to differ without modifying the underlying dataset object. The naive alternative — overwriting subset.dataset — causes the Subset's indices to point into a wrongly-sized dataset, raising an IndexError. Training images receive augmentation. Validation and test images receive only deterministic resize and normalization.

Per-batch metrics — training loss, accuracy, gradient norm — are logged to W&B at each step via wandb.log(..., step=self.global_step). At epoch level, mean gradient norm and gradient clipping ratio are additionally reported. The clipping ratio is the fraction of batches where the gradient norm exceeded the clip threshold.

TABLE I
Training Hyperparameters

Parameter	Value
Optimizer	SGD (Nesterov)
Learning Rate	1×10^{-2}
Momentum	0.9
Weight Decay	5×10^{-4}
Scheduler	Cosine Annealing + Warm-Up
Warm-Up Epochs	10
$T_{\max}$	matched to epoch count
$\eta_{\min}$	1×10^{-4}
Max Epochs	200 (3 in practice)
Batch Size	128
Gradient Clip Norm	1.0
Early Stopping Patience	20

Training ran for 3 epochs. Google Colab session limits constrained this. The configured maximum was 200. T_max and epoch count were overridden via command-line at runtime. Results are discussed in Section VI.

1) Learning Rate Schedule: A cosine annealing schedule with linear warm-up is used. Warm-up spans epochs 0 to $T_w - 1$. During warm-up, the learning rate increases linearly:

$$\eta_t = \eta_{\text{base}} \cdot \frac{t+1}{T_w} \tag{4}$$

After warm-up, cosine decay applies:

$$\eta_t = \eta_{\min} + \frac{\eta_{\text{base}} - \eta_{\min}}{2} \left(1 + \cos\left(\pi \cdot \frac{t - T_w}{T_{\max} - T_w}\right)\right) \tag{5}$$

Early instability is suppressed by the warm-up phase. The learning rate settles toward $\eta_{\min}$ near convergence.

2) Gradient Clipping: Gradient norm clipping is applied after each backward pass. Threshold: $\tau = 1.0$. Exploding gradients are thereby prevented:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1,\ \frac{\tau}{\|\mathbf{g}\|_2}\right) \tag{6}$$

3) Early Stopping: Training halts if validation accuracy fails to improve for 20 consecutive epochs. The best checkpoint is written to disk at each new peak. It stores model weights, optimizer state, scheduler state, and best accuracy. At training completion, this checkpoint is reloaded before test evaluation. Test metrics reflect the best generalization point, not the final epoch.

E. Experiment Tracking

Training runs are logged via Weights & Biases (W&B) [7]. Per-batch metrics: training loss, accuracy, gradient norm. Per-epoch metrics: validation loss and accuracy, learning rate, mean gradient norm, gradient clipping ratio. Test loss and accuracy are logged as summary metrics at run end. The full TRAINING_CONFIG — optimizer kwargs, scheduler kwargs, dataloader configs — is serialized into the W&B run config.

V. Real-Time Inference Pipeline

At inference time, frames are captured from a live webcam feed. Each frame is processed as follows:

1) Hand Detection: MediaPipe processes the frame. Per-hand landmark lists and bounding regions are returned.
2) Stability Check: Euclidean motion between consecutive frames is computed. Motion exceeding 8 pixels flags the hand as unstable. Classification is skipped.
3) Crop and Pad: The highest-confidence bounding box crop is padded to a square aspect ratio. Padding uses edge replication (cv2.BORDER_REPLICATE). Black-border padding introduces distributional shift; edge replication avoids this.
4) Classification: The crop is resized to 224×224 and normalized. It is passed through the SLD model. Softmax probabilities are computed. A prediction is accepted only if confidence exceeds 0.50.
5) Temporal Voting: A majority-vote buffer of size 10 is maintained. A letter is emitted only when 70% or more of the buffer agrees. Isolated mis-classifications do not propagate.
6) Cooldown: A 1-second cooldown is enforced. The same letter cannot be emitted repeatedly in rapid succession.

A. FastAPI Inference Server

A lightweight FastAPI [8] server exposes the inference pipeline as a REST endpoint. JPEG frames are accepted from any HTTP client. Structured JSON predictions are returned.

Endpoint: POST /predict accepts a multipart-encoded JPEG image. The response carries:

- letter — stable, voted prediction (A–Z, del, nothing, space). null if not yet stable.
- confidence — softmax probability of the top class.
- bbox — bounding box as $[x, y, w, h]$ in pixel coordinates.
- handedness — left or right, from MediaPipe.
- motion — Euclidean motion magnitude in pixels. Used for stability gating.
- landmarks — 21 normalized hand landmark coordinates. Used for frontend skeleton rendering.
- error — status string (no hand, hand moving, low confidence, not stable yet, cooldown) when no letter is emitted.

Global state is managed with threading.Lock. This covers the prediction buffer, last emitted letter, and cooldown timestamp. The lock is required for concurrent async workers. CORS is enabled for all origins.

B. Browser Frontend

A single-page HTML/JavaScript frontend interfaces with the FastAPI server. Client-side MediaPipe runs via the Tasks Vision JS SDK at 60 fps. The hand skeleton overlay renders with zero latency at this rate. Frames are forwarded to the backend at approximately 8 fps for classification.

Letter emission uses a dwell-based mechanism. A predicted letter must remain stable for 1.8 s before being appended to the transcript. Subsequent repetitions of the same letter require 3.6 s. Brief hand holds do not produce spurious emissions.

VI. Results and Discussion

Training was conducted on a Google Colab T4 GPU. Session time limits constrained fine-tuning to 3 epochs. The configured maximum was 200. Results below reflect this constrained run.

The model was trained for 3 epochs. Batch size: 128. Optimizer: SGD with Nesterov momentum. Scheduler: cosine annealing with linear warm-up, as described in Section IV. Final performance metrics are summarized in Table II.

TABLE II
Training and Evaluation Results

Split	Loss	Accuracy (%)
Train (epoch 3)	0.0474	99.03
Validation	0.0099	99.86
Test	0.0096	99.80

Test accuracy reached 99.80% at epoch 3. Validation accuracy at epoch 1 was 64.41%. By epoch 3 it reached 99.86%. Training was terminated well short of the configured limit.

These numbers require qualification. The ImageNet-pretrained backbone carries a strong prior. The dataset is constrained: controlled backgrounds, consistent framing, one hand per image. High accuracy under these conditions does not constitute a claim of general robustness.

W&B logs at the final epoch: mean gradient norm of 1.64, gradient clipping ratio of 0.77. The clipping constraint was active. The optimization trajectory had not yet flattened. Training to full convergence would likely yield marginal gains on this dataset; its value would be greater for harder, out-of-distribution evaluation.

The primary challenge for deployment is distribution shift. The training set does not represent varied skin tones, uncontrolled lighting, or cluttered backgrounds. The two-stage architecture partially addresses this. MediaPipe isolates the hand crop before it reaches the classifier, reducing background interference.

A. Colab Cell Output

For reference, the Colab cell output is attached. The full implementation is available at the official GitHub repository here.

VII. Conclusion

This paper presents a real-time ASL alphabet recognition system. The two-stage design — MediaPipe hand detection followed by ResNet-50 classification — achieves 99.80% test accuracy on the Kaggle ASL Alphabet dataset. The system operates at interactive frame rates. Deployment is via a FastAPI inference server and browser-based frontend.

The training pipeline uses warm-up scheduling, Nesterov SGD, gradient clipping, and early stopping. A dwell-based temporal emission mechanism and majority-vote buffer stabilize letter output during live use. The system addresses a real accessibility problem. It is a functional baseline, not a ceiling.

Future directions:

- Fine-tuning MediaPipe on a dedicated hand detection dataset for improved localization.
- Upgrading the backbone to ResNet-152 or EfficientNet for higher classification capacity.
- Extending to dynamic gestures and full ASL words via temporal models (LSTM, Transformer).
- Mobile deployment for broader accessibility reach.

References

[1] World Health Organization, "Deafness and hearing loss," WHO Fact Sheets, 2021. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, 2016, pp. 779–788.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE CVPR, 2016, pp. 770–778.

[4] S. Mitra and T. Acharya, "Gesture recognition: A survey," IEEE Trans. Syst., Man, Cybern. C, Appl. Rev., vol. 37, no. 3, pp. 311–324, 2007.

Fig. 1. FastAPI /predict endpoint response for a live hand frame, showing letter, confidence, bounding box, handedness, motion, and landmark fields.

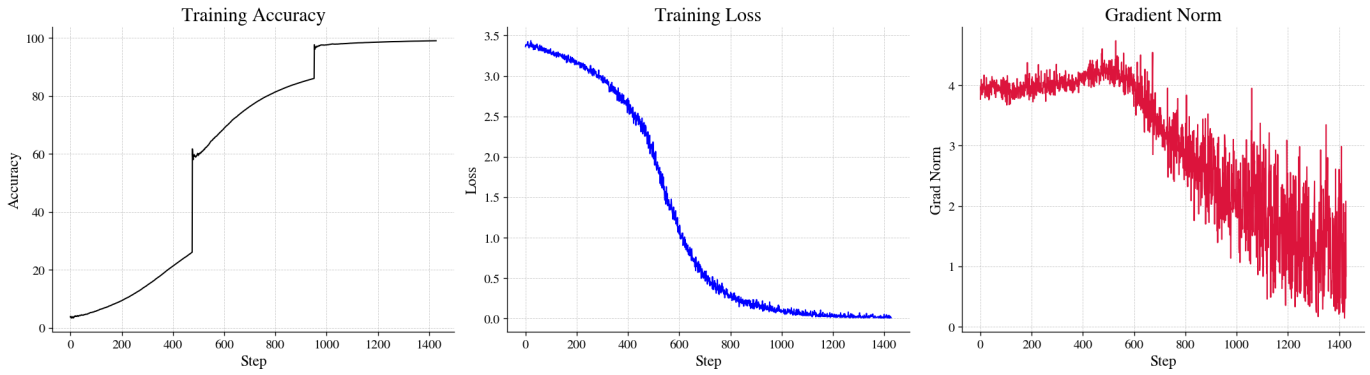

Fig. 2. Training and validation loss/accuracy curves across steps along with gradient norm, logged via Weights & Biases.

[5] L. Pigou, S. Dieleman, P.-J. Kindermans, and B. Schrauwen, "Sign language recognition using convolutional neural networks," in Proc. ECCV Workshops, 2015.
[6] A. Akash, "ASL Alphabet," Kaggle, 2018. [Online]. Available: https://www.kaggle.com/datasets/grassknoted/asl-alphabet
[7] L. Biewald, "Experiment tracking with weights and biases," 2020. [Online]. Available: https://www.wandb.com
[8] S. Ramírez, "FastAPI," 2018. [Online]. Available: https://fastapi.tiangolo.com